

AI Engineering Playbook

How to ship features with AI agents — without shipping the slop.

This playbook is for engineering teams using [ForgeAI](#). It explains the practices behind the workflow, not just how to use the tools. It is grounded in what has demonstrably worked at scale, drawing on published experience from Stripe, Anthropic, Ramp, Google, and independent research across 40,000+ real-world AI engineering workflows.

The Central Insight

"The walls matter more than the model." — Stripe Engineering

AI coding agents are amplifiers. They magnify existing team strengths and dysfunctions equally. A codebase with good test coverage, clear conventions, and tight CI/CD produces excellent agent output. A codebase without these produces more of the same problems, faster.

This playbook describes how to use ForgeAI and how to maintain the engineering infrastructure that makes it work.

What AI Agents Are Good At

- Writing boilerplate and scaffolding from clear specs
- Finding and fixing isolated bugs where test coverage exists
- Generating test cases from acceptance criteria
- Documenting code patterns they can read
- Running structured, repetitive transformations (migrations, refactors with explicit rules)

What They Are Not Good At

- Making architectural decisions without written context
- Working in undocumented, highly-coupled codebases
- Knowing when they are wrong (they fill the context with plausible-looking output if instructions are vague)

- Knowing your domain rules if those rules aren't written down

What this means for you: You must understand the problem before you delegate it. Agents extend your engineering capacity — they do not replace your engineering judgment.

Core Practices

These are ordered by impact. The first three account for the majority of quality gains reported across organisations.

1. Always give agents a way to verify their own work

This is the single highest-leverage practice. Without verification, the only feedback loop is you reading the output. With tests or a build check, the agent can run them, see failures, and self-correct before it reports done.

In practice:

- Write acceptance criteria before asking for implementation
- Pass existing test suites to implementation agents explicitly
- Require agents to run linting and type checks before reporting completion

ForgeAI does this: Kofi (Test Engineer) writes and confirms all tests are failing before Rashidi (Engineer) writes a single line of implementation. Rashidi runs the full test suite before reporting done. This is enforced by Jabari (Orchestrator) — it cannot be skipped.

2. Red/Green TDD — every time

Write failing tests first. Confirm they fail. Then implement.

The failure confirmation step is the one most commonly skipped. If you don't confirm that the tests fail, you risk creating tests that already pass — meaning the tests aren't testing anything, and the implementation that follows has no real constraint.

```
Test Engineer → writes tests → confirms ALL FAILING ← critical step
Engineer → implements → confirms ALL PASSING
```

Teams using strict red/green TDD with AI agents report 40–60% fewer regressions compared to teams that let agents write tests and implementation together.

3. Spec before code

Write a clear spec before any implementation starts. The spec is the primary tool for controlling agent output quality. A vague spec produces vague code. A spec with measurable acceptance criteria produces code you can verify.

A complete spec includes:

- What the feature does (functional requirements)
- What success looks like — measurable (GIVEN/WHEN/THEN)
- What is explicitly out of scope
- Quality targets: coverage, complexity, performance thresholds

ForgeAI does this: Imani (Analyst) produces `design.md` through a five-question Q&A. Jabari (Orchestrator) validates it for completeness before any other agent is allowed to proceed. The spec gates all subsequent work.

4. Clear context between unrelated tasks

The context window is your primary constraint. Performance degrades as it fills with corrections from earlier tasks, unrelated code, and stale instructions.

- Use `/clear` (Claude Code) between unrelated tasks
 - Start a fresh session for each new feature
 - Never let a long session accumulate corrections that don't apply to the current task
 - Use subagents for research (they explore in their own context, report summaries)
-

5. Commit after each checkpoint

Treat git as your undo history. Commit after every gate:

- After tests are written (failing)
- After implementation (passing)
- After QA fixes

Any mistake is then reversible with a single reset. Context windows get stale; git commits don't.

6. Read the output

Treat AI-generated code as code from a capable junior developer who doesn't know your codebase. Read it. Test it. Never ship what you cannot explain.

This is not optional. Across every study and case, the highest-quality outcomes come from engineers who remain in the review loop, not from those who treat agent output as authoritative.

Using ForgeAI

Installation

```
npx forgeai-workflow init
```

Choose your tool (Claude Code / GitHub Copilot / Both) and project type. The CLI installs agents, commands, `CLAUDE.md`, and `AGENTS.md` into your project.

Two paths

Full workflow — Jabari (Orchestrator) coordinates all phases with gate enforcement.

Tool	Command
Claude Code	<code>@orchestrator</code> or <code>/forge-orchestrate</code>
GitHub Copilot	<code>@orchestrator</code> or <code>/forge/orchestrate</code>

Standalone — Invoke any agent directly for a specific task.

Tool	Example
Claude Code	@analyst, @test-engineer, @qa
GitHub Copilot	@analyst, @test-engineer, @qa

Workflow phases

Phase	Agent	What happens	Gate
0	Amina (Codebase Docs)	Scans existing code; produces docs + health scan	—
1	Imani (Analyst)	Requirements Q&A → design.md	Spec completeness validation
2	Zuberi (Architect)	System design → ADRs + diagrams	—
3	Zuri (Designer)	UI screen specs → docs/ux-specs.md	—
4	Kofi (Test Engineer)	Written test plan → failing tests	ALL TESTS FAILING
5	Rashidi (Engineer)	Implementation → all tests passing	ALL TESTS PASSING + complexity check

Phase	Agent	What happens	Gate
5.5	Neema (QA)	Browser testing via Playwright → health score	Critical/high bugs fixed
6	Faraji (DevOps Azure)	CI/CD pipeline + infrastructure + deployment	Deployed + healthy

Gates are binary. A checkpoint clears only on the exact confirmation message. Not on "should be fine." Not on estimation. Not on partial completion.

Gate	Exact confirmation required
Tests written	☒ Tests written – all failing
Implementation done	☒ Implementation complete – all tests passing
Browser QA done	☒ Browser QA complete
Deployed	☒ Deployed and running

The STOP pattern

Every agent waits for your response before proceeding. Nothing advances until you reply. Use this time to read the output carefully, ask questions, and adjust scope before saying yes.

Change Reports

Before any agent creates or modifies a file, it shows a Change Report listing:

- What will be created or changed
- What will not be touched
- Any risks

Reply **yes** to proceed. Reply **no** to stop and redirect. **Read every Change Report before approving.** The approval exists because agents can be wrong about scope.

Prompting Guide

What makes a prompt effective

Ineffective: "Add tests for the auth module."

Effective: "Write failing tests for `src/auth/verifyToken.ts`. Cover: valid token returns user object, expired token throws `TokenExpiredError`, malformed token throws `JsonWebTokenError`. Follow the test patterns in `src/auth/__tests__/parseToken.test.ts`. Confirm all tests are failing before you report done."

The rule: A prompt that would give a senior engineer enough information to start works for an agent. A prompt that would leave a senior engineer with questions will leave the agent making assumptions.

Structure for implementation tasks

1. **Point to the code** — file paths and function names, not descriptions
 2. **Reference existing patterns** — "follow the pattern in `UserRepository.ts`"
 3. **State the verification step** — "run `npm test -- --testPathPattern=auth` after implementing"
 4. **Describe symptoms, not solutions** — for bugs, describe what's broken and how to reproduce it; let the agent reason about the fix
-

For large features: interview first

Ask the agent to interview you before it writes a spec. This is more effective than writing the spec yourself, because the agent asks questions you wouldn't think to answer.

I want to build [feature]. Interview me – ask about requirements, edge cases, technical constraints, and what success looks like. Keep asking until you have

enough to write a complete spec. Then write it to `design.md`.

Start a fresh session to implement from the spec. Clean context, focused on execution.

Anti-hallucination: the ICE method

For tasks where accuracy matters — security, financial logic, data migrations:

- Instructions — specific ask, no open-ended interpretation
- Constraints — "only use what is in the retrieved docs", "don't infer, ask if unsure"
- Escalation — "if you're not certain, say so — don't fill in the gap"

Root cause vs symptoms

When filing a bug with an agent:

Weak: "the login is broken"

Strong: "Login returns 401 for valid credentials. Error in `server/logs/app.log` is `JWT: signature verification failed`. The token is issued by `authController.login` and verified by `middleware/auth.js`. Find the root cause. Don't suppress the error."

Quality Targets

These are the defaults written into `design.md` by Imani (Analyst). Override by telling Imani during the requirements phase.

Metric	Default	Blocker
Test coverage	≥ 85% branch	—
Cyclomatic complexity	≤ 10 per function	≥ 20 blocks the implementation gate

Metric	Default	Blocker
Static analysis	0 critical findings	Blocks merge

These targets are binding. Rashidi (Engineer) runs a complexity scan after all tests pass. Functions at or above 20 must be refactored before the gate clears.

Anti-Slop Contract

All ForgeAI agents enforce these prohibitions. Apply the same checks when you review agent output:

- 1. **No obvious comments** — if the code reads clearly, the comment adds noise, not value
- 2. **No defensive code not in surrounding patterns** — null guards and try/catch blocks that don't appear elsewhere in the file are not safety; they're clutter
- 3. **No type workarounds** — any, unsafe casts, `@ts-ignore` without a specific explanation
- 4. **No pattern drift** — don't introduce patterns that aren't already in the file being modified
- 5. **No over-engineering** — the simplest solution that makes the failing tests pass

These are the first five things to look for when reviewing a diff.

Common Workflows

New feature from scratch

`@orchestrator`

Tell Jabari what you're building. Follow each phase with approval at each Change Report.

Adding a feature to an undocumented codebase

`@codebase-docs`

Let Amina document the codebase and run a health scan first. Then:

| @orchestrator

Plan the feature on the documented base. Agents have full context; they won't invent assumptions about architecture.

Fixing a bug

| @engineer

The build fails with this error: [paste exact error and stack trace].
Fix the root cause. Run `[test command]` after fixing to confirm no regressions.

Writing failing tests for a specific function

| @test-engineer

Write failing tests for [function in file].
Cover: [list of cases].
Follow test patterns in [example test file].
Confirm all tests fail before reporting done.

Running QA on a deployed feature

| @qa
S

Neema runs standard Playwright browser testing, produces a scored health report, and offers to fix critical and high-severity bugs with Change Reports.

Architecture health scan

| @codebase-docs
H

Amina scans for structural problems: high coupling, oversized files, circular dependencies, agent-unfriendly module structures. Returns critical, warning, and healthy areas with specific recommendations.

Team Patterns

Fresh-context code review

After implementation, open a new session (fresh context — no sunk-cost bias from the implementation) and ask it to review the diff:

```
Review the changes in this branch. Look for: edge cases not covered by tests,
code that doesn't match existing patterns in the modified files, and any
violations of the anti-slop contract.
```

A reviewer who didn't write the code catches things the writer won't.

Parallel feature work

Each engineer runs ForgeAI on a separate branch. Sessions don't share context. Run as many parallel feature streams as you have engineers — the workflow is fully isolated per branch.

Sharing prompts

Effective prompts for recurring tasks are team assets. When you find one that works consistently, commit it to `docs/prompts/`. Name it by task type, not date. Review quarterly — model behaviour changes.

Onboarding new engineers

New engineers should:

1. Read this playbook
2. Run `@codebase-docs H` on the codebase (health scan — understand what's healthy and what's not before touching anything)
3. Run a first feature through the full ForgeAI workflow with an experienced engineer present
4. Read every Change Report and every gate confirmation for the first few features before approving

Infrastructure Quality

Agents will produce better output when these exist:

Investment	Why it matters for agents
High test coverage	Agents can verify their own work without relying on you
Fast, reliable CI	Agents get feedback quickly; loops don't stall
Existing documentation	Agents read patterns before writing; fewer invented conventions
Small, well-named modules	Agents find what they need in under 10 seconds; no wrong-file edits
Explicit environment setup	Agents don't guess at env vars, ports, or test commands

Invest here before scaling agent usage. The research is consistent: infrastructure quality is the strongest predictor of agent output quality, more than model capability or prompt sophistication.

Anti-patterns to Avoid

The approval machine

Rubber-stamping every Change Report without reading it. ForgeAI asks for approval because agents can be wrong about scope. Read the report, check what will be touched, verify the approach fits your codebase.

Vibe coding

Asking an agent to implement a feature you don't fully understand yourself. 73% of organisations using natural-language prompts without code comprehension reported quality problems (GitLab, 2025). You must understand the problem to direct the agent.

Context overload

Running long sessions that accumulate corrections from earlier tasks. Use `/clear` between tasks. Start fresh sessions for new features. The longer the session, the more earlier instructions lose weight against newer context.

Skipping the failure confirmation

Writing tests, then immediately handing to the engineer without confirming they fail. If the tests already pass, the engineer has nothing to implement — and you don't know if the tests are testing anything real. This is the most common quality failure in TDD with agents.

Delegating architecture

Asking an agent to make system design decisions without constraints. Agents produce plausible-looking architecture but don't know your team's cost model, your operational reality, or your non-negotiable constraints. Use Zuberi (Architect) to produce and document decisions — but you make the tradeoffs.

Mocked tests

Agents default to mocking when testing is hard. Mocked tests often pass when the real integration fails. CLAUDE.md explicitly prohibits mocking without sign-off. When you see mocks in a test written by an agent, question whether they hide integration risk.

Quick Reference

Agent roster

Agent	Name	Claude Code	Copilot	Best for
Orchestrator	Jabari	@orchestrator	@orchestrator	Full workflow
Codebase Docs	Amina	@codebase-docs	@codebase-docs	Docs, health scans
Analyst	Imani	@analyst	@analyst	Requirements, PRDs
Architect	Zuberi	@architect	@architect	System design, ADRs
Designer	Zuri	@designer	@designer	UI specs, user flows
Test Engineer	Kofi	@test-engineer	@test-engineer	Failing tests (RED)
Engineer	Rashidi	@engineer	@engineer	Implementation (GREEN)
QA	Neema	@qa	@qa	Browser testing
DevOps Azure	Faraji	@devops-azure	@devops-azure	CI/CD, Azure infra

Claude Code slash commands

Command	Phase
<code>/forge-orchestrate</code>	Full workflow
<code>/forge-requirements</code>	Phase 1 — requirements
<code>/forge-architecture</code>	Phase 2 — architecture
<code>/forge-design</code>	Phase 3 — UI specs
<code>/forge-testing</code>	Phase 4 — failing tests
<code>/forge-implementation</code>	Phase 5 — implementation
<code>/forge-deployment</code>	Phase 6 — CI/CD + deploy
<code>/forge-document-codebase</code>	Standalone — codebase docs

Key project files

File	Purpose
<code>design.md</code>	Source of truth — spec, quality targets, acceptance criteria
<code>CLAUDE.md</code>	Claude Code workspace instructions (loaded every session)

File	Purpose
AGENTS.md	Cross-tool instructions (Cursor, Codex, Jules, Copilot, Claude)
docs/architecture.md	System design and ADRs
docs/ux-specs.md	UI screen specifications
.qa/report-*.md	QA reports with health scores and bug lists

Research Grounding

The practices in this playbook are drawn from:

- Anthropic — internal Claude Code workflows and agentic coding trends report (2026)
- Stripe — Minions architecture: 1,300+ AI-authored PRs/week with "Blueprints" workflow
- Ramp — 80% reduction in incident investigation time via MCP-connected agents
- Faros AI — analysis of 10,000+ developers: AI increases PR volume by 98% but increases incidents by 242% without quality infrastructure
- GitLab 2025 AI Paradox Survey — 3,266 DevSecOps professionals
- Simon Willison — agentic engineering patterns including red/green TDD
- Manus — context engineering lessons: keep errors in context, attention recitation via todo lists
- Thoughtworks — spec-driven development patterns
- McKinsey 2025 — only 5.5% of organisations see real returns; high performers redesign workflows rather than just add tools

The consistent finding across all sources: **individual developer throughput increases reliably; team-level quality degrades without deliberate workflow redesign.** This playbook is the workflow redesign.